

Web Programming Course — Academic Report

Relay

A Real-Time Messaging Application

Author

Mukhammadyunus Sheraliev

Date

July 2026

Professor Armando Ruggeri

Table of Contents

1	Introduction	3
1.1	Motivation	3
1.2	Objectives	3
1.3	Project Name	3
2	Technologies Used	4
3	System Architecture	5
3.1	Overview	5
3.2	Why a Proxy?	5
3.3	Frontend Architecture	5
3.4	Backend Architecture	6
4	Authentication System	6
4.1	Registration	6
4.2	Login	6
4.3	Session Verification	7
4.4	Socket Authentication	7
4.5	Route Protection in Next.js	7
4.6	Sequence Diagram – Login	7
5	Real-Time Communication	8
5.1	Why WebSockets?	8
5.2	Socket.IO Lifecycle	8
5.3	Message Lifecycle	9
5.4	Typing Indicators	10
5.5	Presence Grace Period	10
5.6	Multi-Tab Handling	11
6	User Interface and Responsiveness	11
6.1	Layout	11
6.2	Screenshots	12
6.3	Message Virtualization	17
7	Challenges and Solutions	17
7.1	Understanding Server vs. Client Components	17
7.2	Keeping Socket and HTTP Auth in Sync	18
7.3	Optimistic Updates and clientMessageId	18
7.4	Multiple Browser Tabs	18
7.5	State Synchronization	18
8	Database and Data Layer	19
8.1	Schema Overview	19
8.2	Key Design Decisions	19
9	Conclusion	19
10	Appendix	20
10.1	Environment Variables	20
10.2	Socket Events Quick Reference	20

1 Introduction

1.1 Motivation

Real-time communication is everywhere on the web — in customer support widgets, social platforms, collaborative tools. I am really interested in the idea behind every real time app, chat, projects, etc. That's why I chose a real-time app for the project

Relay is a private one-to-one messaging application. It lets two registered users find each other, start a conversation, and exchange messages that appear on both screens in real time. It keeps track of who is online, shows when someone is typing, and marks messages as delivered and read. The goal was to build something that behaves like a real application — not because it has every feature, but because the features it has are implemented end to end and correctly.

1.2 Objectives

The project covers the following learning objectives:

- Understand how full-stack web applications are structured when the frontend and backend are separate services
- Implement HTTP-based authentication with cookies and JWTs
- Build a real-time layer using WebSockets (via Socket.IO) and understand how it differs from the HTTP request-response model
- Use the Next.js App Router, including Server Components, Client Components, Route Handlers, and Server Actions
- Design and query a relational database through an ORM
- Manage client-side state that is driven by server events rather than user actions alone

1.3 Project Name

The application is called **Relay** — a name that fits both the technical function (relaying messages between users over a network) and the intended feel (simple, focused, uncluttered).

2 Technologies Used

Technology	Version	Role
Next.js	16	Frontend framework — App Router, SSR, Route Handlers
React	19	Component model, hooks, <code>useReducer</code>
TypeScript	5 / 6	Static types on both frontend and backend
Tailwind CSS	4	Utility-first CSS
Socket.IO Client	4	WebSocket client
Express.js	5	HTTP API server
Socket.IO	4	WebSocket server
Zod	4	Runtime validation of inputs and env variables
Prisma ORM	7	Type-safe database access, migrations
PostgreSQL	14+	Relational database
bcrypt	6	Password hashing
jsonwebtoken	9	JWT signing and verification
Helmet	8	HTTP security headers
express-rate-limit	8	Auth endpoint rate limiting
@tanstack/react-virtual	3	Virtualized message list

Table 1: Technologies used in Relay

2.0.1 Why Next.js and not a plain React SPA?

A plain React app (Create React App or Vite) requires a separate backend for everything — there is no server-side execution. Next.js blurs this line in useful ways: Server Components can fetch data before anything is sent to the browser, Route Handlers let you write HTTP endpoints inside the same project, and Server Actions allow form submissions to call server-side functions without a client-side fetch call. All of these were used in Relay.

2.0.2 Why Express and not serverless functions?

Socket.IO requires a persistent process — it maintains WebSocket connections that live longer than a single HTTP request. Serverless environments (like Vercel Functions or AWS Lambda) time out after a few seconds, which makes them incompatible with long-lived socket connections. A traditional Express server running in a persistent Node.js process is the straightforward fit.

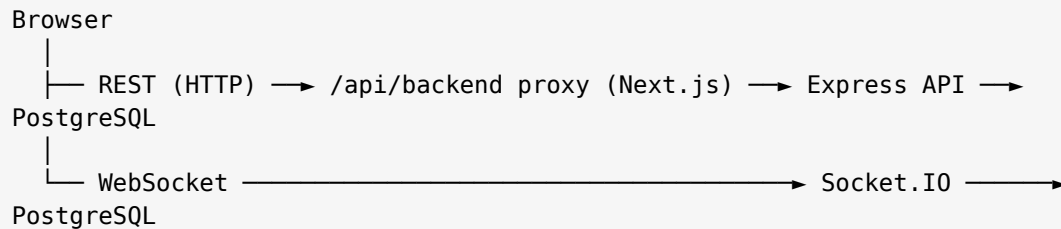
2.0.3 Why TypeScript?

The frontend and backend are separate processes. TypeScript makes it possible to define the shape of an API response once (in `types/api.ts`) and catch mismatches at compile time rather than at runtime. The `ClientToServerEvents` and `ServerToClientEvents` interfaces in `types/socket.ts` give the socket client full type safety for every event it sends or receives.

3 System Architecture

3.1 Overview

Relay runs as two separate Node.js processes: the Next.js frontend and the Express backend. The browser communicates with both, but in different ways.



REST requests go through a Next.js catch-all Route Handler at `/api/backend/[[...path]]`. This handler acts as a transparent HTTP proxy: it forwards the method, headers (including the authentication cookie), and body to the Express server, then streams the response back. The browser only ever talks to one origin.

Socket.IO connects directly from the browser to the backend server. The same `http.Server` instance used by Express handles both HTTP and WebSocket upgrades — no separate port is needed.

Server Components bypass the browser entirely. When a Next.js Server Component needs data for the initial page render, it calls the backend directly (server-to-server HTTP), forwarding the user's cookie. The result is rendered into HTML before anything is sent to the browser.

3.2 Why a Proxy?

Having all browser traffic go through the Next.js proxy solves two problems. First, cookies with `SameSite: Lax` are sent automatically when the request is to the same origin as the page — no CORS configuration needed. Second, the browser does not need to know the backend's address, which simplifies deployment configuration.

3.3 Frontend Architecture

The frontend follows the Next.js App Router pattern. Pages are organized into route groups:



The `(app)/layout.tsx` is a Server Component that checks the user's session before any child renders. If the user is not authenticated, it redirects to `/login` on the server — the browser never sees the protected page at all.

`chat/[conversationId]/page.tsx` fetches the message history on the server and passes it as the initial messages prop to the client component. This means the conversation renders with content immediately, rather than showing a loading spinner.

3.4 Backend Architecture

The backend is organized in three layers:

- **Routes** map URL patterns to controllers and attach middleware. No logic.
- **Controllers** extract inputs from the validated request, call a service function, and return JSON. No database queries.
- **Services** contain all business logic and all Prisma calls. No knowledge of HTTP.

This separation means the socket handlers and the HTTP handlers can share the same service functions — `createMessage` in `conversation.service.ts` is called both from the REST controller and from the socket `message:send` handler.

4 Authentication System

4.1 Registration

When a user submits the registration form, the browser POSTs to `/api/backend/api/auth/register`. The Next.js proxy forwards this to Express. The request body passes through the `validate(registerSchema)` middleware before the controller runs:

- Username: 3–20 characters, alphanumeric and underscores, normalized to lowercase
- Email: valid email format, normalized to lowercase
- Password: at least 8 characters, must contain uppercase, lowercase, digit, and symbol; maximum 72 characters (the bcrypt effective limit)

If validation passes, the service layer hashes the password with bcrypt and creates the user record. If the username or email already exists, Prisma throws an error with code `P2002` (unique constraint violation), which is caught and translated to a 409 response.

4.2 Login

The login form accepts either a username or an email address (the `identifier` field). The backend looks up the user by either field. If found, it compares the submitted password against the stored hash using `bcrypt.compare`. If the comparison passes, it signs a JWT:

```
const token = jwt.sign({ userId: user.id }, env.JWT_SECRET, {
  expiresIn: env.JWT_EXPIRES_IN // default: "1d"
});
```

The token is then set as a cookie:

```
res.cookie("rtc_auth", token, {
  httpOnly: true,      // JavaScript cannot read this cookie
  sameSite: "lax",     // not sent on cross-site POSTs
  secure: env.NODE_ENV === "production",
  maxAge: 24 * 60 * 60 * 1000,
});
```

The browser receives the cookie and will send it automatically on every subsequent request. It cannot read the token value through JavaScript.

4.3 Session Verification

Every protected HTTP endpoint goes through the authenticate middleware:

```
export async function authenticate(req, res, next) {
  const token = req.cookies?.["rtc_auth"];
  if (!token) throw new ApiError(401, "UNAUTHENTICATED", "...");

  const payload = jwt.verify(token, env.JWT_SECRET); // throws if invalid
  const user = await prisma.user.findUnique({
    where: { id: payload.userId },
    select: { id: true, username: true, email: true },
  });

  if (!user) throw new ApiError(401, "UNAUTHENTICATED", "...");
  req.user = user;
  next();
}
```

The middleware verifies both the JWT signature **and** that the user still exists in the database. Deleting a user account immediately prevents further authenticated requests.

4.4 Socket Authentication

The Socket.IO connection requires the same `rtc_auth` cookie. The socket middleware parses it from the handshake headers manually (Express's `cookie-parser` does not apply to socket connections), verifies the JWT, and attaches the user to `socket.user`. Connections without a valid token are rejected before any event handlers run.

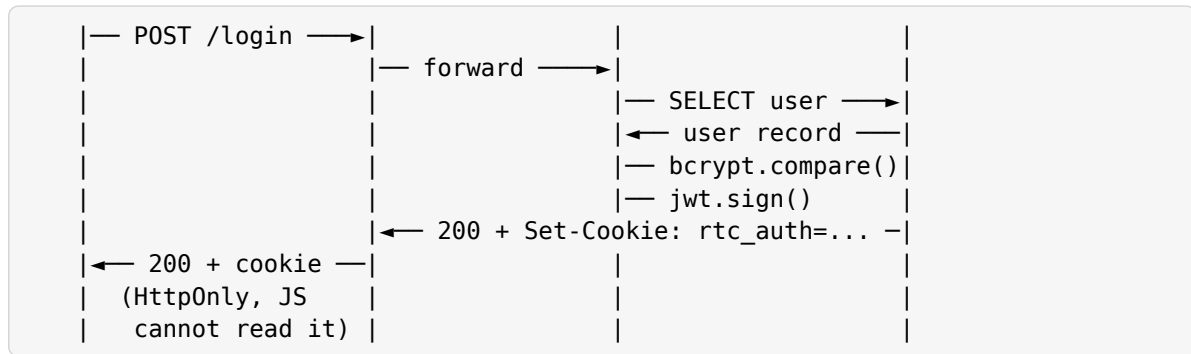
4.5 Route Protection in Next.js

`(app)/layout.tsx` calls `getCurrentUser()` — a cached server function that fetches `/api/auth/me`. If it returns `null`, the layout calls `redirect("/login")`. Because this is a Server Component, the redirect happens before any HTML is sent to the browser. There is no flash of the protected page.

The reverse also applies: `(auth)/layout.tsx` redirects already-authenticated users to `/chat`, so logged-in users cannot navigate back to the login page.

4.6 Sequence Diagram — Login

Browser	Next.js Proxy	Express	PostgreSQL



5 Real-Time Communication

This is the most interesting part of the project, and probably the one that required the most learning.

5.1 Why WebSockets?

HTTP is a request-response protocol. To get new data from a server, the client has to ask for it. For a chat application, polling (sending a request every second to check for new messages) works — but it is wasteful. Every poll is a full HTTP round trip, even when nothing has changed. At any meaningful scale this adds up.

WebSockets solve this by establishing a persistent, bidirectional connection. Once the connection is open, the server can push data to the client at any time without the client asking. Socket.IO is a library built on top of WebSockets that adds useful features: automatic reconnection, a room/namespace abstraction, middleware support, and fallback to HTTP long-polling in environments that do not support WebSockets.

5.2 Socket.IO Lifecycle

5.2.1 Connection

When the ChatShell component mounts (after the page loads), it creates a socket with `autoConnect: false` and then calls `socket.connect()` inside a `useEffect`. The socket connects with `withCredentials: true`, which tells the browser to include cookies in the handshake.

The server's `authenticateSocket` middleware runs synchronously on the connection attempt. If it fails, the connection is rejected. If it passes, `socket.user` is set.

On a successful connection, the server immediately:

1. Adds the socket to the user's personal room (`user:<userId>`)
2. Records the new socket ID in the presence store
3. Updates `isOnline = true` in the database (if this is the user's first socket)
4. Broadcasts `presence:online` to **all** connected users

5.2.2 Room System

Two types of rooms are used:

- **user:<userId>**: a personal room that each user's socket(s) join on connect. Presence events and new-message notifications for the sidebar go here. Because a user might have multiple tabs open, all sockets for a user are in this room simultaneously.
- **conversation:<conversationId>**: a room for a specific conversation. Joined when the client emits `conversation:join`. Used for direct per-conversation targeting.

Most server-to-client events are emitted to user rooms rather than conversation rooms. For example, when a new message arrives in a conversation, the server looks up all participant user IDs and emits `message:new` to each `user:<id>` room. This ensures a user gets the event (for the sidebar unread count) even if they have not opened that conversation.

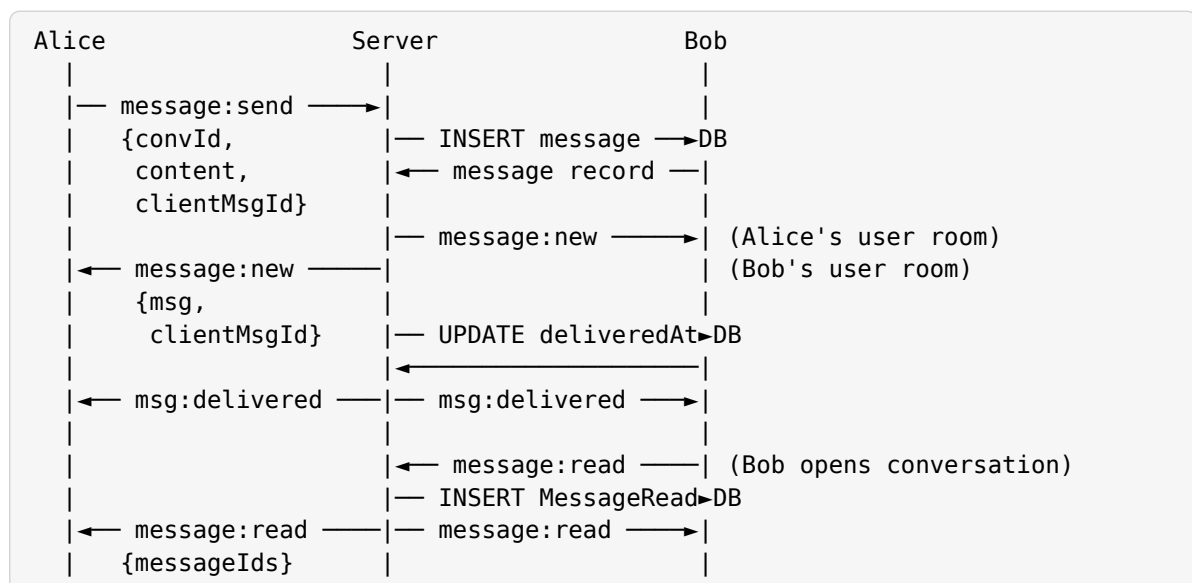
5.2.3 Disconnection

When a socket disconnects, the server removes it from the presence store. If the user has no remaining sockets (they closed all tabs), the server:

1. Updates `isOnline = false` and `lastSeenAt = now()` in the database
2. Broadcasts `presence:offline` with the `lastSeenAt` timestamp to all connected users

5.3 Message Lifecycle

5.3.1 Sequence diagram



5.3.2 Optimistic UI

When Alice presses Enter, `MessagePanel` immediately dispatches a `messageQueued` action with a locally-constructed message object (status: "sending"). This message appears in the chat right away. Simultaneously, the socket event `message:send` is emitted.

When the server confirms the message, it emits `message:new` back to Alice with the same `clientMessageId` she sent. The reducer replaces the pending placeholder with the server-confirmed record:

```
case "messageReceived": {
  return {
    ...state,
    messages: upsertMessage(state.messages, action.message,
      action.clientMessageId),
    // also updates the sidebar preview
  };
}
```

`upsertMessage` filters out both the pending entry (by `clientMessageId`) and any duplicate server record (by `id`), then inserts the new message and re-sorts by timestamp. This handles both the normal case and the edge case where the socket reconnects and the confirmation arrives late.

If the socket is not connected when the user tries to send, the message is immediately dispatched with `messageFailed` and shown with a red outline.

5.3.3 Delivery Receipts

Immediately after saving the message to the database, the server updates `deliveredAt` and emits `message:delivered`. This fires even before the recipient's client has received the message — it confirms database persistence, not browser delivery.

Read receipts are more precise. The client emits `message:read` only when:

- The conversation is currently open
- `document.visibilityState === "visible"` (the tab is actually focused)

This avoids marking messages as read when the user has the tab in the background.

5.4 Typing Indicators

The `MessageComposer` component calls `announceTyping()` on each keystroke:

```
function announceTyping() {
  socket.emit("typing:start", { conversationId });
  if (stopTimer.current) window.clearTimeout(stopTimer.current);
  stopTimer.current = window.setTimeout(() => {
    socket.emit("typing:stop", { conversationId });
  }, 900);
}
```

`typing:start` is emitted on every keystroke. A 900-millisecond debounce timer fires `typing:stop` after the user pauses. The server verifies the user is a participant, then re-broadcasts the event to the other user's room. No state is maintained on the server for typing — the debounce lives entirely on the client.

5.5 Presence Grace Period

When a user disconnects, the frontend does not immediately show them as offline. Instead, a 60-second grace window is applied on the client:

```
const ONLINE_GRACE_PERIOD_MS = 60_000;

export function isUserOnline(user, now = Date.now()) {
```

```
if (user.isOnline) return true;
if (!user.lastSeenAt) return false;
return Math.abs(now - Date.parse(user.lastSeenAt)) < ONLINE_GRACE_PERIOD_MS;
}
```

A `usePresenceClock` hook re-evaluates this every second. This prevents jarring status flickers caused by brief network interruptions or page refreshes.

5.6 Multi-Tab Handling

The presence store on the backend is a `Map<userId, Set<socketId>>`. When a user opens a second tab, a second socket is added to the set. When they close one tab, only that socket is removed. The user only goes offline when the set is empty:

```
export function removeUserSocket(userId, socketId) {
  const sockets = userSockets.get(userId);
  sockets?.delete(socketId);
  if (sockets?.size === 0) {
    userSockets.delete(userId);
    return 0; // caller broadcasts presence:offline
  }
  return sockets?.size ?? 0;
}
```

Without this, closing one tab would incorrectly mark a user offline even though they still have another tab open.

6 User Interface and Responsiveness

6.1 Layout

The main chat layout uses a CSS Grid with two columns on screens wider than the Tailwind md breakpoint (768px):

```
grid-cols-[22rem_1fr]
```

The left column (22rem wide) is the conversation sidebar. The right column takes the remaining space and shows the active conversation.

On mobile, only one panel is visible at a time. The sidebar is hidden when a conversation is active (`hidden md:flex`), and the message panel is hidden on the conversation list page (`hidden md:grid`). A back arrow in the message panel header navigates back to the sidebar. This is all done with Tailwind utility classes — no JavaScript breakpoint detection.

6.2 Screenshots

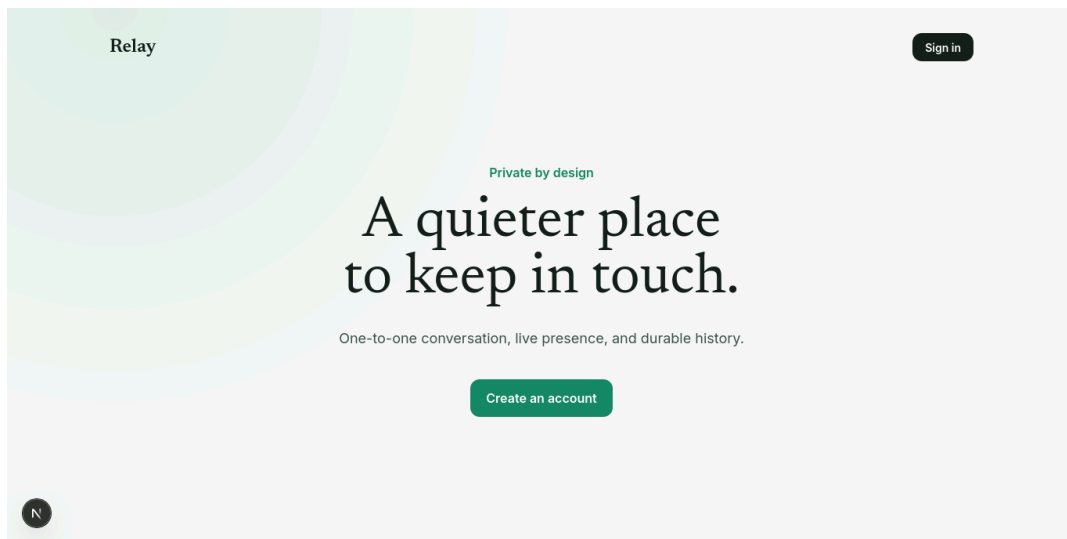


Figure 1: **Figure 1 — Home page.** The landing page introduces Relay with a minimal design. A single call-to-action leads to account creation.

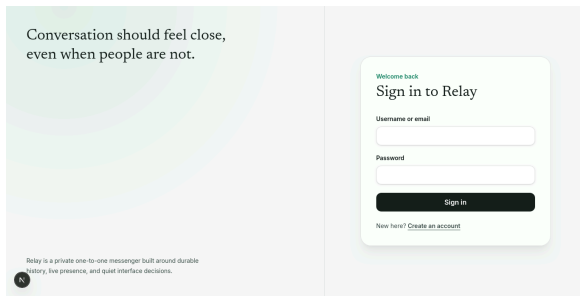


Figure 2: **Figure 2 — Login (desktop).** Two-column layout: tagline on the left, form on the right.

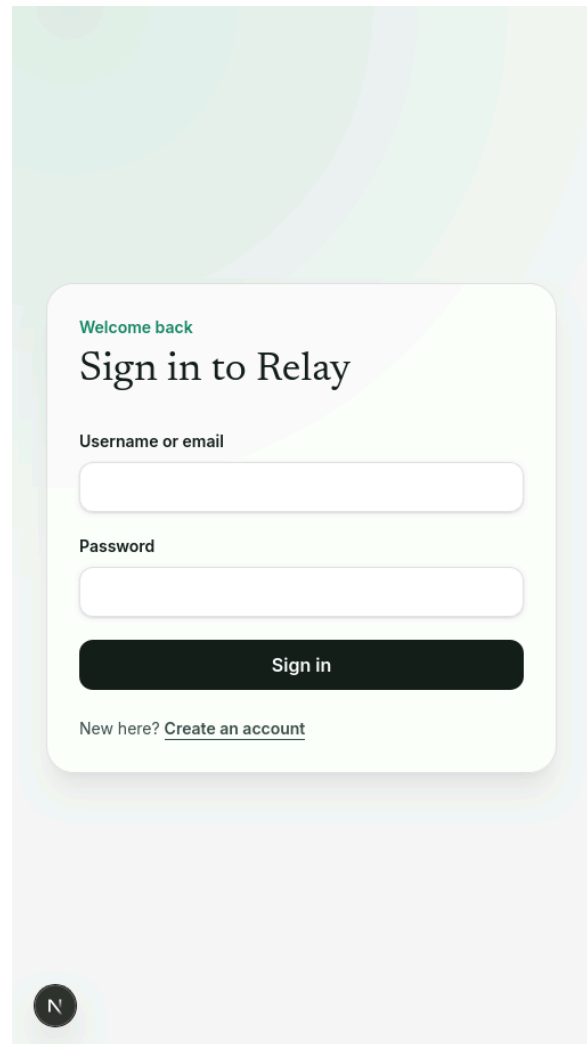


Figure 3: **Figure 3 — Login (mobile).** Single column, form centered.

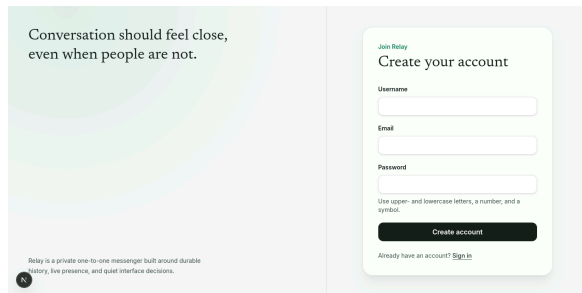


Figure 4: **Figure 4 — Register (desktop)**. Password hint shown below the field.

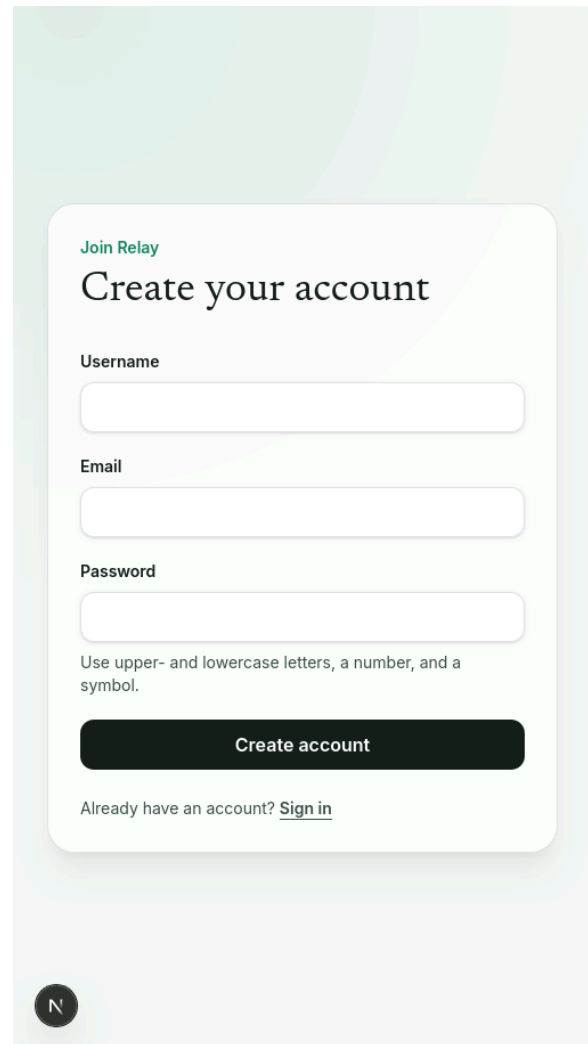


Figure 5: **Figure 5 — Register (mobile)**. Same form, single-column layout.

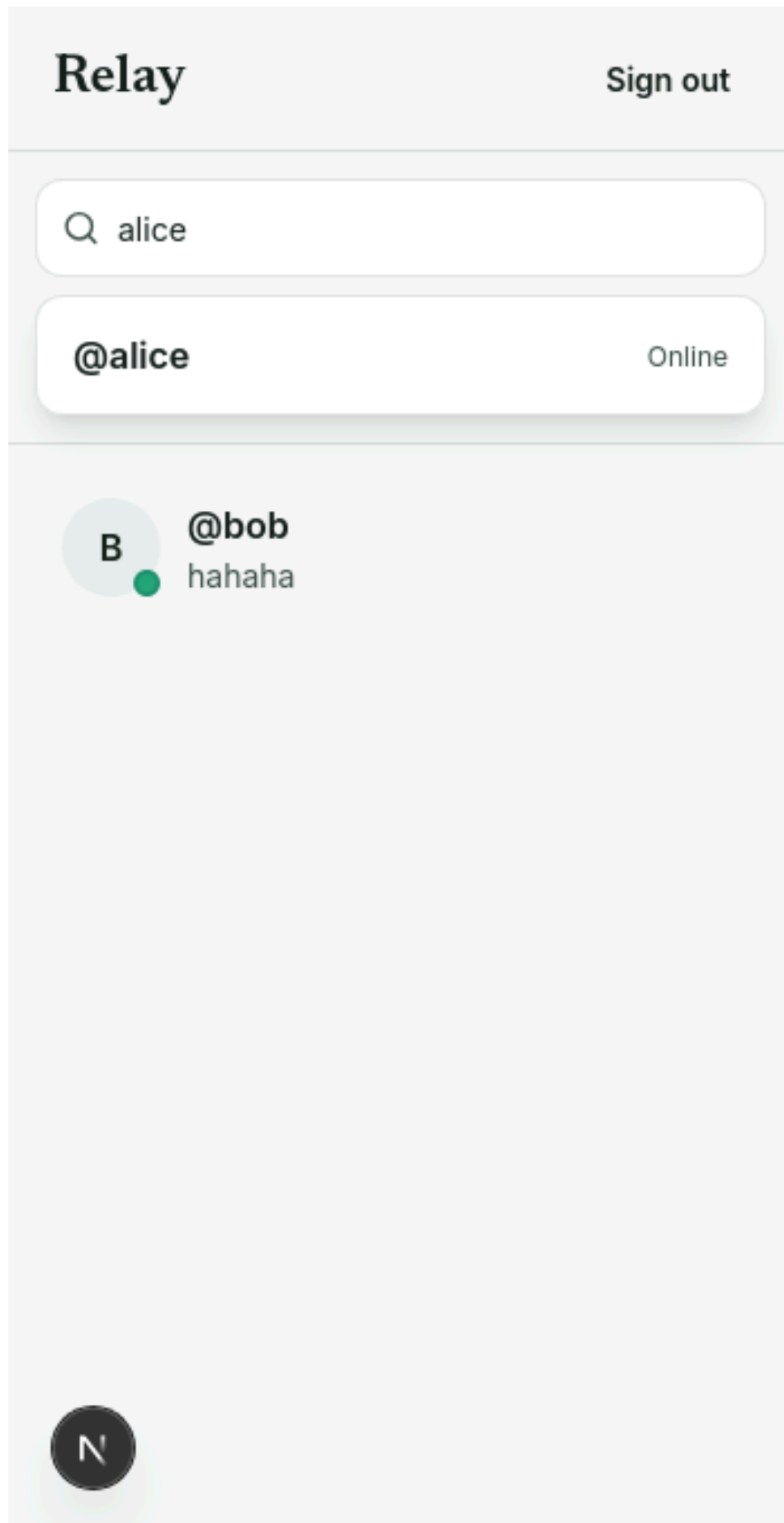


Figure 6: **Figure 6 – User search.** The search bar at the top of the sidebar filters users by username. Selecting one creates or navigates to an existing conversation.

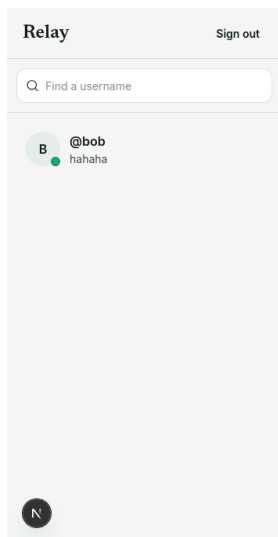


Figure 7: **Figure 7 — Chat main view.** The sidebar lists conversations with avatar initials, online indicator (green dot), and the most recent message preview. No active conversation is selected.

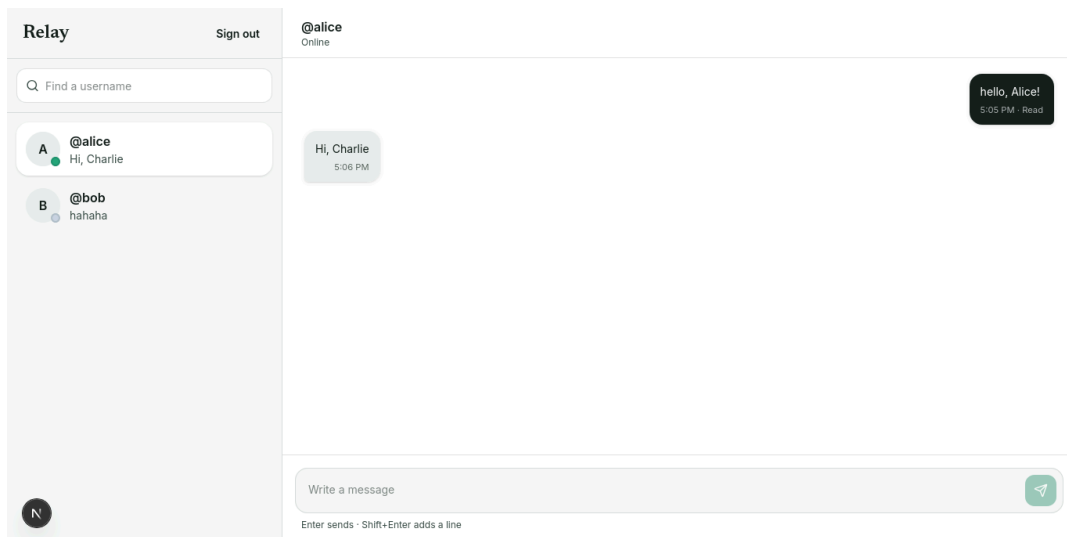


Figure 8: **Figure 8 — Active conversation.** Outgoing messages align right (dark background). Incoming messages align left (light background). The header shows real-time online status. Each outgoing message shows a delivery and read status below it.

6.3 Message Virtualization

For long conversations, rendering every message as a DOM element is expensive. The `MessageList` component uses `@tanstack/react-virtual` to render only the rows currently visible in the scroll container, plus an 8-row overscan buffer:

```
const virtualizer = useVirtualizer({
  count: messages.length,
  getScrollElement: () => scrollRef.current,
  estimateSize: () => 76, // estimated row height in px
  overscan: 8,
  getItemKey: (index) => messages[index].id,
});
```

Each virtual item is positioned absolutely using `translateY`. `useLayoutEffect` scrolls to the bottom whenever the message count increases (a new message arrived).

7 Challenges and Solutions

7.1 Understanding Server vs. Client Components

The App Router’s Server/Client split is not immediately obvious. The constraint is that Server Components cannot use `useState`, `useEffect`, or any browser API. Client Components can, but they cannot be `async` and cannot import server-only modules.

The pattern that eventually made sense: Server Components are data-fetching shells. They load the initial state and pass it as props to Client Components, which take over from there. `ChatShell` is a Client Component because it needs to manage a socket, but the pages that render it (`chat/page.tsx`, `chat/[conversationId]/page.tsx`) are Server Components that fetch the initial data.

A concrete mistake that cost time: calling `cookies()` from `next/headers` inside a Server Action without `await`. In Next.js 15+, `cookies()` is async. Forgetting the `await` causes the cookie header to be missing on backend requests, which produces 401 errors that look completely unrelated to the actual problem.

7.2 Keeping Socket and HTTP Auth in Sync

The Express `authenticate` middleware uses `req.cookies` (parsed by `cookie-parser`). Socket.IO does not go through Express middleware, so `cookie-parser` does not run for socket connections. The socket auth middleware has to parse the cookie header from `socket.handshake.headers.cookie` manually.

Both code paths verify the same JWT and load from the same `prisma.user.findUnique` call, which means any change to the auth model (e.g., adding user roles) needs to be updated in both places. This is a maintenance concern that a shared utility function would address.

7.3 Optimistic Updates and `clientMessageId`

The optimistic update pattern is straightforward in concept: add the message to the UI immediately, wait for the server to confirm. The tricky part is knowing which pending message the server’s confirmation corresponds to.

The solution is a client-generated UUID (`crypto.randomUUID()`) sent with the socket event as `clientMessageId`. The server echoes this back in the `message:new` event to the sender. The reducer uses this to find and replace the pending placeholder. Without this round-trip of the client ID, the client would not know which pending message to replace — it would add a duplicate instead.

7.4 Multiple Browser Tabs

An early implementation marked a user offline immediately when their socket disconnected. This broke as soon as someone had two tabs open: closing one tab would mark them offline even though the other tab was still connected.

The fix is the presence store — a `Map<userId, Set<socketId>>`. The user goes offline only when the set is empty. This was a conceptual shift from “one user, one socket” to “one user, N sockets” that is easy to miss when writing the initial implementation.

7.5 State Synchronization

The chat state (`conversations`, `messages`, `typingUsers`) is all managed in a single `useReducer`. Every socket event dispatches an action that produces a new state immutably. The advantage of this approach over using multiple `useState` calls is that it keeps all the update logic in one function (`chatReducer`) that can be read independently of the component tree.

The hardest part of the reducer was the `read` action, which has to:

1. Add a `MessageRead` record to the right messages in the `messages` array
2. Decrement `unreadCount` on the right conversation in the `conversations` array
3. Handle the case where the read was by the current user vs. the other user

All three effects need to happen atomically from the client’s perspective — a partial update would cause the UI to show inconsistent data.

8 Database and Data Layer

This section is intentionally brief, as database design belongs to a separate course.

8.1 Schema Overview

User —< ConversationParticipant >— Conversation —< Message —< MessageRead

A `Conversation` links two users through `ConversationParticipant` records. A `Message` belongs to a `Conversation` and has a sender. A `MessageRead` records when a specific user read a specific message.

8.2 Key Design Decisions

One conversation per pair. A `pairKey` column on `Conversation` holds a canonical string: the two user IDs sorted and joined with `:`. This is unique in the database, so two simultaneous requests to create the same conversation will conflict on the unique key and converge on the same record.

Cursor-based pagination. Message history is paginated by taking `limit + 1` rows from the database. If the result has more rows than the limit, there are more pages — the last row's ID becomes the next cursor. This is stable under concurrent inserts (unlike offset-based pagination, which can miss or duplicate rows when the dataset changes between requests).

Prisma ORM. All queries use Prisma, which generates TypeScript types for every query result. This means the service layer's return types are inferred from the database schema rather than written manually.

9 Conclusion

Relay does what it sets out to do. Two users can register, find each other, and have a real-time conversation with all the expected affordances: typing indicators, read receipts, online presence, and a message history that does not reset when you refresh the page.

Building it clarified several things that were only abstract before. The boundary between the HTTP layer and the WebSocket layer is cleaner when you understand that they are genuinely separate channels with separate authentication paths. The difference between Server and Client Components in Next.js is not a stylistic choice but a practical one — data fetching on the server avoids entire categories of loading states and race conditions. And managing state that is driven by asynchronous server events is genuinely different from managing state driven by user input — the `chatReducer` approach turned out to be the right tool for it.

The main thing the project is missing is tests. The `chatReducer` is a pure function that would be easy to unit test, and the service layer functions would benefit from integration tests against a real database. That is the most obvious thing to add next.

Outside of that, the natural extensions are things that require new infrastructure: Redis for horizontal socket scaling, object storage for file attachments, and push notifications for reaching users when the browser is closed. These are real engineering problems that were understood well enough to describe clearly, even if they are not implemented here.

10 Appendix

10.1 Environment Variables

10.1.1 Backend

Variable	Default	Description
DATABASE_URL	required	PostgreSQL connection string
FRONTEND_ORIGIN	required	Allowed CORS origin (full URL)
JWT_SECRET	required	Signing secret (≥ 32 characters)
JWT_EXPIRES_IN	1d	Token lifetime
BCRYPT_COST	12	bcrypt work factor (10–14)
PORT	4000	Server port
NODE_ENV	development	Environment mode

10.1.2 Frontend

Variable	Description
BACKEND_URL	Backend URL for server-to-server calls (not exposed to browser)
NEXT_PUBLIC_SOCKET_URL	Backend URL for Socket.IO connection (sent to browser)

10.2 Socket Events Quick Reference

10.2.1 Client → Server

Event	Purpose
conversation:join	Join a conversation's socket room
message:send	Send a message
message:read	Mark messages as read
typing:start	Notify typing started
typing:stop	Notify typing stopped

10.2.2 Server → Client

Event	Purpose
message:new	New message arrived (with optional <code>clientId</code>)
message:delivered	Message persisted to database
message:read	Messages marked as read by a user
typing:start	Another user started typing
typing:stop	Another user stopped typing
presence:online	A user connected

<code>presence:offline</code>	A user disconnected (all tabs closed)
<code>socket:error</code>	Handler-level error